

Full Stack Development with MERN

Project Documentation — Customer Care Registry

1. Introduction

Project Title: Customer Care Registry

Team Members:

[Team Member 1 Name]	– Role (e.g. Frontend Developer)
[Team Member 2 Name]	– Role (e.g. Backend Developer)
[Team Member 3 Name]	– Role (e.g. AI Integration / QA)
[Team Member 4 Name]	– Role (e.g. UI/UX & Documentation)

2. Project Overview

Purpose

Customer Care Registry is a role-based web platform that unifies how a business receives, tracks, and resolves customer complaints. It replaces disconnected phone/email/spreadsheet-based support with a single system where customers raise and track tickets, agents and managers resolve them with AI-assisted categorization, and admins oversee the whole operation through analytics, audit logs, and configurable settings.

Features

- Role-based access for Super Admin, Support Manager, Support Agent, and Customer, each with their own dashboard modules.
- Customer management: add, edit, and view customer profiles (Standard / Enterprise / VIP) with full interaction and complaint history.
- Complaint / ticket management: raise complaints with attachments, auto-generated ticket IDs, assignment, status lifecycle, and a full action timeline.
- AI-assisted complaint categorization, priority/department suggestion, and reply drafting via the Google Gemini API, with a keyword-based fallback when the AI is unavailable.
- Interaction logging across Phone Call, Email, Chat, Visit, WhatsApp, and Video Call channels.
- Feedback collection with rating, comments, satisfaction level, and AI-derived sentiment.
- Knowledge Base with searchable FAQ / Solution / Article entries and view-count tracking.
- Analytics dashboard: complaint trends, department and agent performance, and satisfaction metrics.
- Audit logging of key user actions, and system settings for SLA hours and auto-escalation.

3. Architecture

Frontend

Built with React 19 and TypeScript, bundled with Vite and styled with Tailwind CSS v4. The app (src/App.tsx) is a single-page application that manages the logged-in user, active view, and role-based navigation via a Sidebar and Header component. Charts are rendered with Recharts, icons with lucide-react, and transitions with the motion library. Key views include the Analytics Dashboard, Ticket Center, Customer Hub, Knowledge Base, Audit Logs, and System Settings, each conditionally shown based on the current user's role.

Backend

Built with Node.js and Express 4 in TypeScript, run directly with tsx in development (server.ts). The server exposes a REST API under /api, secured by a custom token-authentication middleware (authenticateToken) and a role-based access control middleware (requireRole). In development, the Express server also creates a Vite dev server in middleware mode to serve the React app; in production it serves the built static assets.

Database

Data is persisted through a lightweight JSON file-based data layer (server/db.ts), which exposes typed accessor methods (getUsers, getCustomers, getComplaints, getInteractions, getFeedbacks, getKnowledgeArticles, getAuditLogs, getSettings, etc.). The data model covers Users, Customers, Complaints (tickets), Interactions, Feedback, Knowledge Base Articles, Audit Logs, and System Settings. This layer is intentionally abstracted behind simple functions so it can be swapped for MongoDB (via Mongoose) or another database without changing the Express route handlers or the frontend API contract.

4. Setup Instructions

Prerequisites

- Node.js (v18 or later recommended) and npm.
- A Google Gemini API key (optional — the app falls back to keyword-based categorization without it).
- (Optional, for future DB migration) A MongoDB instance / connection string, if replacing the JSON data layer.

Installation

1. Clone the repository:

```
git clone <repository-url>
cd customer-care-registry
```

2. Install dependencies:

```
npm install
```

3. Create a .env file in the project root and set the required environment variables:

```
GEMINI_API_KEY=your_gemini_api_key_here
NODE_ENV=development
```

4. Start the application (this runs the Express server, which also serves the Vite-powered React frontend):

```
npm run dev
```

The application will be available at the URL printed in the terminal (default: http://localhost:3000).

5. Folder Structure

Client (src/)

```
src/
├─ App.tsx           # Root component: routing, role-based views, state
├─ main.tsx          # React entry point
├─ index.css          # Tailwind CSS entry
├─ types.ts           # Shared TypeScript types (User, Customer, Complaint, etc.)
├─ components/
│   ├─ Header.tsx     # Top navigation bar
│   ├─ Sidebar.tsx    # Role-based side navigation
│   ├─ CustomerDetail.tsx # Customer profile / history view
│   └─ TicketDetail.tsx # Ticket / complaint detail view
```

Server (server/ and root)

```
server.ts           # Express app, all REST API route handlers
server/
├─ db.ts             # JSON file-based data layer & data model types
├─ ai.ts              # Gemini AI integration: categorization, reply, sentiment
vite.config.ts       # Vite configuration (dev server, HMR)
tsconfig.json        # TypeScript configuration
```

6. Running the Application

Frontend

The React frontend does not run as a separate process; it is served by the Express server through a Vite middleware-mode dev server during development, and as static built assets in production.

Backend

```
# Development (Express + Vite middleware, with hot reload)
npm run dev

# Production build (bundles the React app and the Express server)
npm run build
npm run start
```

7. API Documentation

All endpoints are prefixed with /api and return JSON. Endpoints marked “Authenticated” require an Authorization: Bearer <token> header; “Role-restricted” endpoints additionally require the user to hold one of the listed roles.

7.1 Authentication

Method	Endpoint	Auth / Role	Description
POST	/api/auth/login	Public	Authenticates a user with email and password; returns a token and the user profile.
POST	/api/auth/register	Public	Registers a new user (defaults to Customer role); auto-creates a linked customer profile for Customer registrations; returns a token and user profile.
GET	/api/auth/me	Authenticated	Returns the currently authenticated user's profile.

7.2 Customer Management

Method	Endpoint	Auth / Role	Description
GET	/api/customers	Authenticated	Lists customers (filterable), scoped to what the caller's role is allowed to see.
POST	/api/customers	Super Admin, Support Manager	Creates a new customer record.
PUT	/api/customers/:id	Authenticated	Updates an existing customer record.
DELETE	/api/customers/:id	Super Admin	Deletes a customer record.

7.3 Complaint / Ticket Management

Method	Endpoint	Auth / Role	Description
GET	/api/complaints	Authenticated	Lists complaints/tickets with filters (search, status, priority, category, department, assignedAgentId); results are scoped by role (customers see only their own tickets, agents see assigned/unassigned tickets).
GET	/api/complaints/:id	Authenticated	Retrieves a single ticket; customers are blocked from viewing tickets that are not their own.
POST	/api/complaints/ai-analyze	Authenticated	Runs AI categorization on a title/description before filing, returning suggested category, priority, and department.
POST	/api/complaints	Authenticated	Creates a new ticket, optionally running AI-based routing (runAiRouting), and auto-assigns the customer ID for Customer-role callers.
PUT	/api/complaints/:id	Authenticated	Updates a ticket (status, assignment, timeline notes, etc.).
DELETE	/api/complaints/:id	Super Admin	Deletes a ticket record.
POST	/api/complaints/:id/ai-reply	Authenticated	Generates an AI-assisted reply suggestion for the ticket.

7.4 Interaction Logging

Method	Endpoint	Auth / Role	Description
GET	/api/interactions	Authenticated	Lists logged customer interactions (calls, emails, chats, visits, WhatsApp, video calls).
POST	/api/interactions	Super Admin, Support Manager, Support Agent	Logs a new customer interaction.

7.5 Feedback

Method	Endpoint	Auth / Role	Description
GET	/api/feedbacks	Authenticated	Lists feedback records.
POST	/api/feedbacks	Authenticated	Submits feedback for a resolved ticket; sentiment is derived via the AI module.

7.6 Knowledge Base

Method	Endpoint	Auth / Role	Description
GET	/api/kb	Public	Lists published knowledge base articles.
GET	/api/kb/:id	Public	Retrieves a single knowledge base article and increments its view count.
POST	/api/kb	Super Admin, Support Manager	Publishes a new knowledge base article.

7.7 Analytics, Audit & Settings

Method	Endpoint	Auth / Role	Description
GET	/api/analytics	Authenticated	Returns aggregate metrics and chart data (complaint trends, department/agent performance, satisfaction).
GET	/api/audit-logs	Super Admin, Support Manager	Lists audit log entries recording key user actions.
GET	/api/settings	Authenticated	Retrieves current system settings (SLA hours, auto-escalation).
PUT	/api/settings	Super Admin	Updates system settings.

Example: POST /api/auth/login

Request body:

```
{
  "email": "agent@example.com",
  "password": "*****"
}
```

Example response:

```
{
  "token": "YWdlbnRAZXhhbXBsZS5jb206U3VwcG9ydCBBZ2VudDp1c3JfMTIz",
  "user": {
    "id": "usr_123",
    "email": "agent@example.com",
    "fullName": "Jane Doe",
    "role": "Support Agent",
    "department": "Customer Relations",
    "status": "Active"
  }
}
```

8. Authentication

The API uses a lightweight, self-contained token scheme rather than a signed JWT library. On successful login or registration, the server Base64-encodes a string of the form email:role:userId and returns it as the token. The frontend stores this token and sends it on every subsequent request as an Authorization: Bearer <token> header.

On each request, the authenticateToken middleware decodes the token, looks up the matching user by ID and email, and rejects the request (401/403) if the token is missing, malformed, the user cannot be found, or the user's account status is 'Inactive'. Role-based authorization is then enforced per-route by the requireRole(...) middleware, which checks the authenticated user's role against an allow-list before the handler runs (e.g. only Super Admin can delete a customer or update system settings).

Note: this token is not cryptographically signed like a standard JWT (no signature/expiry claims), which is acceptable for a project/demo deployment but should be upgraded to a signed, expiring JWT (or a session-based auth provider) before any production use with real customer data.

9. User Interface

The interface is organized around a persistent Sidebar (role-filtered navigation) and Header, with the main content area switching between the following screens based on the selected module and the logged-in user's role:

- Analytics Dashboard — metric cards and charts (Super Admin, Support Manager, Support Agent).
- Ticket Center — ticket list with filters, and a detailed ticket view (TicketDetail.tsx) showing timeline, AI suggestions, and actions (all roles, scoped per role).
- Customer Hub — customer list and a detailed customer view (CustomerDetail.tsx) with interaction/complaint history (Super Admin, Support Manager, Support Agent).
- Knowledge Base — searchable article list (all roles).
- Audit Logs — chronological action log (Super Admin, Support Manager).
- System Settings — SLA and escalation configuration (Super Admin).

[Insert screenshots or a short screen-recording/GIF of each of the above screens here.]

10. Testing

Testing strategy for this project combines:

- Manual functional testing against the User Acceptance Testing (UAT) document, covering registration/login, ticket lifecycle, AI categorization (including the offline fallback path), assignment, SLA due dates, feedback, knowledge base search, and role-based access restrictions.
- Static type-checking via the TypeScript compiler (npm run lint, which runs tsc --noEmit) to catch type errors across the client and server code before runtime.
- [Optional / future work] Automated testing tools such as Jest or Vitest for unit tests on server/ai.ts heuristics and server/db.ts data functions, and React Testing Library for component tests.

11. Screenshots or Demo

[Insert screenshots of the Analytics Dashboard, Ticket Center, Customer Hub, Knowledge Base, and Settings screens here, or a link to a hosted demo / recorded walkthrough.]

12. Known Issues

- Authentication tokens are Base64-encoded but not cryptographically signed or time-limited (see Section 8); a token cannot currently be invalidated server-side before its holder logs out client-side.
- The data layer is a JSON file store (server/db.ts), which is convenient for development but is not suited to concurrent write-heavy production use or horizontal scaling.
- AI-based categorization quality depends on the Gemini API being configured and reachable; very short complaint descriptions can still produce a generic fallback category (see UAT bug BG-002).
- Knowledge Base search behavior and multilingual feedback sentiment analysis have known accuracy limitations (see UAT bugs BG-004 and BG-005).

13. Future Enhancements

- Migrate the data layer from the JSON file store to MongoDB (with Mongoose) or another production-grade database, without changing the REST API contract.

- Replace the Base64 token scheme with signed, expiring JWTs (or a managed auth provider), including refresh tokens and server-side revocation.
- Add automated unit and integration tests (Jest/Vitest, React Testing Library) and set up CI to run npm run lint and tests on every change.
- Add real-time updates (e.g. WebSockets) so agents and customers see ticket status changes instantly without refreshing.
- Expand AI capabilities: automatic ticket summarization, multilingual sentiment analysis, and smart routing based on agent workload.
- Add file-storage integration (e.g. S3-compatible storage) for attachments instead of inline Base64 storage, to reduce payload size and improve performance at scale.

References

<https://react.dev/>

<https://expressjs.com/>

<https://vitejs.dev/>

<https://ai.google.dev/gemini-api/docs>

<https://www.mongodb.com/docs/>